QUESTION 1

- Write a program to print “HELLO!” on the screen.
- In your submission include a file named exercise2.asm containing the solution for this exercise.

QUESTION 2

- Write a program that can convert the user input character to UPPERCASE like below
- Example:
  - ENTER A LOWER-CASE LETTER: a
  - IN UPPERCASE IT IS: A
- Hint: look at ASCII table; to convert a character to uppercase, subtract **32** or **20h** from it
- In your submission include a file named exercise2.asm containing the solution for this exercise

Steps:
; Show a prompt to the user to input a lower-case number
; Take the single character input from the user
; Print a new line
; Convert input to uppercase
; Print the converted character
; Return control to DOS

QUESTION 3

- Write a program that takes a single digit number as input from the user (1..5).
- Makes sure that the input is between 0 and 5.
- Print whether the number is odd or even.

Steps:
; Show a prompt to the user to input 1 single digit number between 1 and 5
; Take the single character input from the user
; Subtract 48 from input to convert the ASCII character to its decimal form
; Make sure that the number is between 1 and 5; otherwise, go back to taking input again
(use CMP and Jump operations here)
; Print a new line
; Check if the number is odd or even
(use CMP and Jump operations here)
; Print ‘o’ for odd and ‘e’ for even
; Return control to DOS

QUESTION 4

Draw a detailed flowchart showing how the Intel 8086-based IBM PC boots.
Your flowchart should include:

- The role of the **BIOS**.
- How the **boot program** is loaded.
- How control is finally transferred to **DOS** (the operating system).

QUESTION 5

You are tasked with analyzing the organization and interaction of the components in the Intel IBM PC 8086 microprocessor. It is broadly divided into two components, EU and BIU, each with multiple registers (general, segment, offset etc.) and logic units organized between them. Based on your understanding of this, answer the following questions.

- **Discuss the role of the EU.** In your answer, include what "EU" stands for, what it contains (include the relevant register types), its major functions, where it stores intermediate data, and how it communicates with the BIU.

- **Discuss the role of the BIU.** In your answer, include what BIU stands for, what the BIU contains (including the types of registers), and the major function of the BIU (refer to specific registers within the BIU).

- **Define instruction prefetching on the 8086.** When does prefetching occur and when does it not occur?

IMPORTANT DOS FUNCTIONS

TO DISPLAY A SINGLE CHARACTER

```
MOV AH, 02h        ; DOS service: display a single char
MOV DL, '?'        ; character to print goes in DL
INT 21h            ; call interrupt
```

TO PRINT A NEW LINE

```
MOV AH, 02h
MOV DL, 0Dh        ; Carriage Return (CR)
INT 21h

MOV DL, 0Ah        ; Line Feed (LF)
INT 21h
```

TO DISPLAY A STRING

```
LEA  DX, MSG       ; DS:DX -> "$"-terminated string
MOV  AH, 09h
INT  21h
```

TO READ A CHARACTER/ SINGLE DIGIT FROM TERMINAL/USER

```
MOV AH, 1          ; DOS service: read character (echoed)
INT 21h            ; returns ASCII code in AL
MOV BL, AL         ; save the character for later
```

RETURN CONTROL TO OPERATING SYSTEM

```
MOV AX, 4C00h
INT 21h
```

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[ENG OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	|
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

HINTS

Convert from ASCII to numeric value

AUB AL, '0' ; now AL = numeric value

Convert from numeric to ASCII value

ADD AL, '0' ; now AL = numeric value

TEMPLATE

```
.MODEL SMALL ; Makes sure that we have only 1 data and 1 code segment
.STACK 100H ; Allocates 256 bytes of memory for stack
.DATA
      B_ARRAY DB 10h, 20h, 30h ; Allocates 3 bytes in memory
      MSG DB 'THIS IS A MESSAGE$'
      MASK EQU 8
.CODE
MAIN PROC
      ; Activate data segment
      MOV  AX, @DATA
      MOV  DS, AX

      ;
      ASSEMBLY INSTRUCTIONS
      ;
MAIN ENDP

END MAIN
```